Activity: Converting your prototype into an application

Practice Assignment • 30 min

Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 80%. We keep your highest score.

Next item →

Converting your prototype into an application

Overview

In this activity, an AI Studio prototype is converted into a functional application that runs on a local computer. The code generated by AI Studio will be improved with two important features: enhanced security for the API key and error handling to make the application more reliable. These are essential skills for building professional, AI-powered tools.

Scenario

A client has approved a "Travel Agent Generator" prototype created in AI Studio. The next task is to turn that prototype into a simple, dependable tool that the client can run from their computer. The goal is to improve the application's security and reliability to meet the standards for a real-world application.

Before you begin

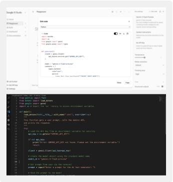
Make sure you have the following ready:

1. **Your prototype:** You will use the optimized prompt: `Create a 3 day itinerary for a trip to Japan`.
2. **Your Gemini API key:** You will need the API key you previously created.
3. **A code editor:** A simple text or code editor like Visual Studio Code is recommended.
4. **Python or JavaScript:** Make sure you have either Python installed on your computer.

Steps

Step 1: Export and set up the environment

Two phases of local codebase



The first phase is to move the code from AI Studio to a local file and prepare the development environment.

1. **Select a prototype:** In Google AI Studio, open a prompt created in the previous module (e.g., a summarizer, a quiz generator, or a tagline creator).
2. **Export the code:** Select the **Get Code** button. Choose a programming language (Python or JavaScript) and copy the entire code snippet.
3. **Set up local files:**
   - Create a new folder on the computer for this project.
   - Inside the new folder, create a new file. Name it `app.py` for Python
   - Paste the copied code into this new file.
4. **Install libraries:** Open a terminal or command prompt, navigate to the project folder, and install the required library.
   - For Python: `pip install google-generativeai`

1. **A developer has finished prototyping a text summarizer in AI Studio. They now want to add a feature that saves each summary to a local text file, a function that is not available directly within the AI Studio interface. What is the essential first step they must take to add this custom feature? Select the best answer.** 1 / 1 point

● Use the "Get Code" feature in AI Studio to export the prototype as a script in a programming language like Python.
○ Take a screenshot of the AI Studio interface to document the prototype's settings.
○ Continue to refine the prompt in AI Studio to ask the model to save the file.
○ Find a different AI model that has built-in file-saving capabilities.

Nice work

Correct! Exporting the code is the necessary step to move from the limited AI Studio environment to a full development environment where custom features like file saving can be added.

2. **After pasting the exported code into `app.py`, a developer tries to run the script but receives a `ModuleNotFoundError: No module named 'google.genai'`. What is the most likely cause of this error? Select the best answer.** 1 / 1 point

● The developer forgot to install the required client library using pip.
○ The Gemini model is currently offline.
○ The API key was entered incorrectly in the .env file.
○ The developer is using the wrong programming language.

Nice work

Correct! Code exported from AI Studio relies on external libraries (like google-genai). You must install these in your local environment before the script can run successfully.

Step 2: Implement secure API key management and setup

The exported code includes the API key written directly in the file. This is a major security risk. The key must be removed from the code and loaded securely from an **environment variable**.

Set the environment variable: Set the `GEMINI_API_KEY` environment variable on the computer in the same directory as the app.py file.

Python snippet

`GEMINI_API_KEY=AIzaSy.........`

Add the following code before copying the code from Get Code in AI Studio in app.py. Paste the code you exported afterwards

```
app.py

import os
from pathlib import Path
from dotenv import load_dotenv
from google import genai
from google.genai import types

# Load .env located next to this script
load_dotenv(dotenv_path=Path(__file__).with_name(".env"))
 # Copy exported code here
```

3. **After exporting their AI Studio project, a developer notices their API key is written directly in the Python code file: `API_KEY = "aIzaSy..."`. They plan to share this project with a colleague for review. What is the most important change they should make before sharing the code? Select the best answer.** 1 / 1 point

○ Save the file in a compressed zip folder before sending it.
○ Add comments to the code explaining what the API key is used for.
○ Change the variable name from `API_KEY` to something less obvious, like `SECRET_STRING`.
● Remove the key from the code and modify the script to load it from a secure environment variable instead.

Nice work

Correct! This is a critical security practice. It separates the secret key from the code, preventing it from being accidentally exposed when the code is shared or stored in a public place.

4. **You have successfully moved your API key into a `.env` file to keep it out of your main code. If you decide to upload your project to a public website like GitHub, what is the best way to ensure the `.env` file (and your secret key) stays private? Select the best answer.** 1 / 1 point

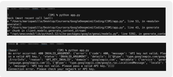
● Use a .gitignore file to tell the system to ignore the .env file during the upload.
○ Delete the .env file and re-type the API key into the code before uploading.
○ Change the name of the file to "secrets.txt" so it is harder to find.
○ Don't worry about it; public sites automatically hide files that start with a dot.

Nice work

Correct! A `.gitignore` file is a standard tool that prevents sensitive files like `.env` from being uploaded to public sharing sites while allowing the rest of your code to be shared.

Step 3: Implement robust error handling

Before and after of handling failure gracefully



```
)                              C1M1 % python app.py
back (most recent call last):
e "/Users/mariopadilla/Desktop/Coursera/GoogleDeepmind/Coding/C1M1/app.py", line 53, in <module>
enerate()
e "/Users/mariopadilla/Desktop/Coursera/GoogleDeepmind/Coding/C1M1/app.py", line 43, in generate
or chunk in client.models.generate_content_stream(
e "/opt/anaconda3/lib/python3.12/site-packages/google/genai/models.py", line 5392, in generate_conte
```

```
(base)                              C1M1 % python app.py
An error occurred: 400 INVALID_ARGUMENT. {'error': {'code': 400, 'message': 'API key not valid. Ple
ss a valid API key.', 'status': 'INVALID_ARGUMENT', 'details': [{'@type': 'type.googleapis.com/goog
.ErrorInfo', 'reason': 'API_KEY_INVALID', 'domain': 'googleapis.com', 'metadata': {'service': 'gene
language.googleapis.com'}}, {'@type': 'type.googleapis.com/google.rpc.LocalizedMessage', 'locale': '
', 'message': 'API key not valid. Please pass a valid API key.'}]}}
Connection error. Please check your network or API key.
```

Real-world applications can face network problems or API issues. The code must be prepared to handle these errors without crashing. This is done by wrapping the API call in a `try...except` block.

1. **Identify the API call:** Locate the line of code that sends the request to the Gemini API (e.g., `response = model.generate_content(...)`).
2. **Wrap in an error-handling block:** Implement a `try...except` block around the call.

Python implementation snippet

```
def generate():
try:
    # This is the original line from your exported code

    # Print the successful response here
  print(chunk.text, end="")

except Exception as e:
    # This block runs ONLY if the API call or connection fails
    print(f"An error occurred: {e}")
    print("Connection error. Please check your network or API key.")
if __name__ == "__main__":
    generate()
```

5. **A developer's new application, created from an AI Studio prototype, works perfectly on their fast office network. However, when a user with an unstable internet connection tries it, the application crashes with an error. What programming technique should the developer use to prevent the crash and instead show a friendly message like "Connection error, please try again"? Select the best answer.** 1 / 1 point

○ Optimize the prompt to make the model's response shorter, which will use less data.
○ Add more `if/else` statements to check the user's input before making the API call.
● Wrap the API call inside a `try...except` block.
○ Use a `for` loop to make the API request five times in a row.

Nice work

Correct! This structure is designed specifically for this purpose. It allows the program to "try" an operation that might fail. If it does, the "except" block runs, preventing a crash and allowing the developer to handle the error gracefully.

Step 4: Test the application

Now it is time to test the improved application.

1. Open a terminal or command prompt.
2. Navigate to your application folder.
3. Set your environment variable (if you haven't already).
4. Run the application with the command: `python app.py`

Test a few scenarios:

- **Success:** Does it run correctly and print a travel itinerary?
- **Failure (bad key):** Try running it with an incorrect API key (or no key set). Does it show your friendly error message instead of crashing?
- **Failure (no internet):** Try turning off your Wi-Fi and running it. Does it handle the connection error gracefully?

Success checklist

Your activity is complete when:

- The application runs successfully from the computer's terminal.
- The API key is no longer written directly in the code file.
- The code loads the API key from an environment variable.
- The application prints a helpful message instead of crashing when an API error occurs.

Key points

- **Security:** Removing the API key from the code and using an environment variable is the **most critical production practice** for protecting credentials.
- **Reliability:** The `try...except` block is essential for **graceful failure**, ensuring your application doesn't crash due to external factors like network latency or API issues.

Exemplar solution (optional)

Want to compare your work to a finished example? You can view a complete, commented version of the final `app.py` file here

```
# pip install -U google-genai python-dotenv

import os
from pathlib import Path
from dotenv import load_dotenv
from google import genai
from google.genai import types

# Load .env located next to this script
load_dotenv(dotenv_path=Path(__file__).with_name(".env"))

# To run this code you need to install the following dependencies:
# pip install google-genai

import base64
import os
from google import genai
from google.genai import types


def generate():
    try:
        # This is the original line from your exported code
        client = genai.Client(
            api_key=os.environ.get("GEMINI_API_KEY"),
        )

        model = "gemini-3-flash-preview"
        contents = [
            types.Content(
                role="user",
                parts=[
                    types.Part.from_text(text="Create a 3 day itinerary for a trip to Japan."),
                ],
            ),
        ]
        generate_content_config = types.GenerateContentConfig(
            thinking_config=types.ThinkingConfig(
                thinking_level="HIGH",
            ),
```

Reflect on your work

- Why is using an environment variable for an API key safer than writing it in the code?
- How does adding a try...except block make an application more user-friendly?
- What other kinds of errors might happen in a real application that would need to be handled?

Next steps

Great job! You have successfully turned a prototype into a more secure and reliable application. You now have a solid foundation for the next module, where you will learn how to deploy your application to the cloud.